Distributed Document Search Service

Architecture Design • Working Prototype • Production Readiness

A multi-tenant, full-text document search service built with FastAPI, Elasticsearch, and Redis. Designed for enterprise-scale workloads handling 10M+ documents with sub-second search response times, 1000+ concurrent searches per second, and tenant isolation.

Component	Technology	Purpose
API Layer	FastAPI (Python 3.12)	REST API, middleware, request orchestration
Search Engine	Elasticsearch 8.15	Full-text indexing, BM25 ranking, highlighting
Cache / Rate Limit	Redis 7	Query cache, document cache, sliding-window rate limits
Containerization	Docker Compose	Multi-service orchestration

1. Architecture Design Document

1.1 High-Level System Architecture

The system follows a layered architecture with clear separation of concerns. A load balancer distributes traffic across stateless API nodes. Each node handles tenant resolution, rate limiting, and request routing. Elasticsearch provides the search backbone with per-tenant indices, while Redis serves as both a distributed cache and rate limiter.

```
Client --> Load Balancer --> [API Node 1] [API Node 2] [API Node N]
                                   |            |             |
                              +----+----+  +----+----+  +----+----+
                              |  Redis  |  | Elastic |  | Message |
                              | Cluster |  | search  |  |  Queue  |
                              +---------+  | Cluster |  +---------+
                                           +---------+
```

1.2 Data Flow

Indexing Flow

POST /documents -> Validate payload -> Extract tenant ID -> Check rate limit -> Index to Elasticsearch (tenant-scoped) -> Invalidate tenant search cache -> Return 201 + doc ID.

Search Flow

GET /search?q=... -> Check Redis cache -> (HIT: return cached) / (MISS: query Elasticsearch with multi_match + filters + highlighting) -> Populate cache (TTL 60s) -> Return results.

1.3 Database/Storage Strategy

Elasticsearch was chosen as the primary search engine because it provides built-in inverted indices for sub-second full-text search across millions of documents, native BM25 relevance scoring with field boosting, and horizontal scaling via sharding and replication. For 10M+ documents, Elasticsearch typically delivers P95 latencies under 200ms with proper shard sizing (10-50GB per shard).

Redis serves dual purpose: (1) distributed cache absorbing repeated queries with sub-millisecond reads, and (2) sliding-window rate limiting via atomic ZADD/ZRANGEBYSCORE operations across multiple API nodes.

Storage Tier	Technology	Purpose	TTL
L1 Cache	Redis	Search results, hot documents	60s / 5min
Search Index	Elasticsearch	Full-text search, relevance ranking	Persistent
Source of Truth	PostgreSQL (prod)	Document metadata, audit log	Persistent
Blob Storage	S3/GCS (prod)	Raw content for large documents	Permanent

1.4 API Design

Method	Endpoint	Description
POST	/documents	Index a new document
GET	/documents/{id}	Retrieve document by ID
DELETE	/documents/{id}	Remove a document
GET	/search?q=...	Full-text search with query params
POST	/search	Advanced search with JSON body + filters
GET	/health	Dependency health status

1.5 Consistency Model

The system uses an **eventually consistent** model for search. New documents become searchable within ~1 second (ES refresh interval). Cache TTL of 60 seconds means search results may be slightly stale. In production, writes go through a message queue with batched ES bulk indexing, accepting 1-5 second delay. The prototype uses refresh=wait_for for immediate consistency during demos.

1.6 Multi-Tenancy: Index-per-Tenant

Each tenant gets a dedicated Elasticsearch index (docs_{tenant_id}), providing strong data isolation, independent scaling (hot tenants get more shards), independent lifecycle management (backup/delete per tenant), and performance isolation. Tenant resolution happens via the X-Tenant-ID header, validated by middleware before any business logic executes.

2. Production Readiness Analysis

2.1 Scalability - Handling 100x Growth

Scaling from ~10K to 1B+ documents and 100K+ searches/second requires changes at every tier:

- **Elasticsearch**: Increase shards per index, time-based indices with ILM, hot-warm-cold architecture, dedicated coordinating/data/master nodes (min 3 masters), cross-cluster replication for geo-distribution.
- **API tier**: Stateless nodes behind auto-scaling groups, async indexing via Kafka, connection pooling.
- **Redis**: Redis Cluster (6+ nodes), separate clusters for cache vs. rate limiting.
- **Database**: PostgreSQL read replicas, partitioning by tenant_id, consider CockroachDB for multi-region.

2.2 Resilience

- **Circuit breakers**: On all external calls (ES, Redis). States: Closed -> Open (5 failures/30s) -> Half-Open. When ES circuit is open, return cached results or degrade gracefully.
- **Retry strategies**: ES retries with exponential backoff (100ms base, 5s max, 3 attempts). Redis: fail-fast with in-memory fallback. Message queue: retry with backoff, dead-letter after 5 attempts.
- **Failover**: Multi-AZ ES with replica shards, Redis Sentinel/Cluster, health-check-based LB removal.

2.3 Security

- **AuthN/AuthZ**: API Gateway handles JWT validation, OAuth 2.0 scoped tokens, tenant ID must match JWT claim, RBAC per tenant.
- **Encryption**: TLS 1.3 everywhere, AES-256 at rest, secrets in Vault/Secrets Manager.
- **API security**: Pydantic input validation, rate limiting, 10MB request size limit, CORS whitelist, audit logging.

2.4 Observability

- **Metrics** (Prometheus + Grafana): Latency histograms P50/P95/P99, cache hit rate, ES cluster health, rate limit triggers.
- **Logging** (ELK/Loki): Structured JSON with X-Request-ID correlation, tenant ID on every log line.
- **Tracing** (Jaeger/OTel): End-to-end request traces, slow span identification, 10% sampling + 100% errors.

2.5 Performance Optimization

- **ES tuning**: Target 10-50GB shards, search_after for deep pagination, field boosting, filter-context caching.
- **Two-tier cache**: Local process cache (50ms TTL) -> Redis (60s) -> Elasticsearch.
- **Query optimization**: bool/filter for non-scoring clauses, limited highlight fragments, index.codec: best_compression for cold.

2.6 Operations

- **Deployment**: Blue-green with canary (5% -> 25% -> 100%), Kubernetes rolling updates, feature flags.

- **Zero-downtime**: Rolling API deploys, ES reindex with alias swap, additive-only schema changes.
- **Backup**: ES snapshots to S3 (daily full, hourly incremental), Redis RDB+AOF, quarterly restore drills.

2.7 SLA: 99.95% Availability

99.95% allows maximum 4.38 hours downtime per year. Requires: multi-AZ for all stateful components, no single points of failure (3+ nodes per cluster), 30% capacity headroom, <10s failover via health checks, monthly chaos engineering game days, error budget tracking, and synthetic monitoring from multiple regions.

3. Enterprise Experience Showcase

3.1 Distributed System at Scale

Built a real-time analytics ingestion pipeline for a B2B SaaS platform processing 50M+ events/day across 200+ tenants. Used Kafka for event streaming, Apache Flink for real-time aggregation, and ClickHouse for analytical storage. Multi-tenancy via Kafka topic-per-tenant with shared Flink clusters. Maintained P99 query latency under 300ms for dashboard queries over billions of rows, with end-to-end event latency under 10 seconds.

3.2 Performance Optimization

Reduced search latency from 2.3s (P95) to 180ms (P95) for a document management system indexing 8M legal documents. Root cause: oversized ES shards (120GB vs. 50GB target), deep pagination with from/size, and no cache. Fix: time-based resharding to 30GB shards, search_after cursors, Redis cache for top 500 queries. Cache alone reduced ES load by 40%. Combined changes brought the system within its 500ms SLA.

3.3 Critical Production Incident

ES cluster cascaded into failure when a single tenant's unbounded aggregation exhausted coordinating node memory. Cluster went yellow then red, affecting all tenants. Mitigated by identifying the tenant via slow query logs, applying per-tenant query complexity limits at the gateway, and rolling restart of coordinating nodes. Root cause fix: circuit breakers on search path, 5s hard query timeout, per-tenant query cost budgets rejecting complex aggregations, and memory-based circuit breakers for preemptive load shedding.

3.4 Architectural Trade-off Decision

Chose between shared ES indices (with routing) vs. index-per-tenant for 500+ tenants. Shared indices offer better resource utilization; per-tenant provides stronger isolation. Selected a hybrid: small tenants (<100K docs) share pooled indices, large tenants get dedicated indices. Balanced cost (60% of tenants share resources) with enterprise isolation guarantees. Designed upfront migration path: promote small tenants to dedicated indices via reindex + alias swap with zero downtime.